Projet : *Potager du roi*

Modélisation et manipulation du graphe routier

Contexte

La principale contrainte du projet réside dans le fait que les livraisons s'effectuent exclusivement à vélo (éventuellement électrique) depuis un dépôt unique. Cela réduit drastiquement la taille du réseau routier pertinent pour le projet, et rend possible le chargement de l'intégralité de son modèle (graphe pondéré) en mémoire vive (RAM) sans recourir à une base de données externe.

Le graphe pondéré, modélisant le réseau routier, sera chargé en mémoire vive à partir des fichiers au format `CSV` fournis.

Description des fichiers CSV

— `data/sommets-enrichis.csv` contient l'ensemble des sommets (intersections de tronçons de route) du réseau routier sous forme de coordonnées cartésiennes exprimées dans le système **Lambert 93**.

Les coordonnées sont exprimées en mètres et sont précédées de l'identifiant du sommet.

```
stud@psl: ~/COURS/JAVA/potager/ $ cat data/sommets-enrichis.csv | head -n
    5
id;x;y
51;642927.50;6853262.20
52;642995.50;6853258.10
68;649834.70;6871087.40
69;649915.10;6871049.20
```

— `data/arcs-enrichis.csv` contient l'ensemble des arcs (routes) du réseau routier. Il contient

1. l'identifiant de l'arc,
2. l'identifiant du sommet source,
3. l'identifiant du sommet cible,
4. le coût du trajet dans le sens aller (de la source vers la cible),
5. le coût du trajet dans le sens retour (de la cible vers la source),
6. la géométrie de l'arc au format WKT (Well-Known Text) [https://en.wikipedia.org/wiki/Well-known_text_representation_of_geometry], constituée dans notre cas uniquement de lignes brisées connexes :

$$\texttt{MULTILINESTRING}\big((x_1\ \ y_1\texttt{,}\cdots\texttt{,}x_n\ \ y_n)\big).$$

Un coût égal à `-1` signifie que le sens correspondant est interdit (sens unique).

```
stud@psl: ~/COURS/JAVA/potager/ $ cat data/arcs-enrichis.csv | head -n 3
id;source;target;cost_s;reverse_cost_s;geometry_wkt
1;354166;775440;4.284247;4.284247;MULTILINESTRING((655369.8 6858639.9,655383.9 6858615.2,655389.5 6858605.2,655401.1 6858589.4))
2;775440;459390;8.638008;8.638008;MULTILINESTRING((655401.1 6858589.4,655471.7 6858492.4))
```

On remarquera que le séparateur utilisé dans les fichiers CSV est le point-virgule `;` et que la première ligne contient les en-têtes des colonnes.

1 Modélisation des structures de données

Pour optimiser le stockage, on privilégie une approche `Data-Oriented Design` sous un format plat, en organisant les enregistrements en colonne plutôt qu'en ligne, au lieu d'utiliser une arborescence d'objets lourde.

L'intérêt d'un enregistrement en colonne est de réduire la fragmentation mémoire.

Les données d'un même type (homogènes) étant contiguës en mémoire se prêtent plus efficacement :

— aux accès mémoire (meilleure utilisation du cache),
— aux traitements vectoriels,
— au calcul parallèle,
— à la compression.

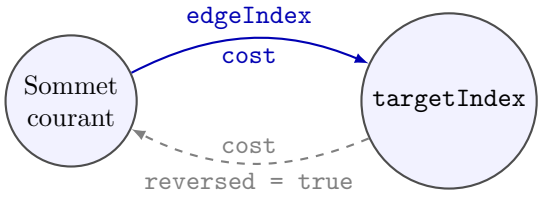
L'inconvénient de cette approche est l'absence d'une structure englobante (encapsulation). L'entité logique étant fragmentée, le maintien de la cohérence des données — notamment la synchronisation des indices entre les différentes listes parallèles est entièrement à la charge du programmeur.

Question 1. Définissez une classe publique, qui n'accepte pas d'héritiers, nommée `RoadGraph` contenant, entre autres,

— Une classe imbriquée statique, le `record Arc`, pour représenter un arc sortant

```
public record Arc (int targetIndex, double cost, int edgeIndex, boolean reversed) {}
```

Le champ `reversed` indique que l'arc parcourt la ligne CSV de la cible vers la source (coût `reverse_cost_s`) ; il servira à orienter la géométrie lors du tracé d'un chemin. Il n'indique pas que l'arête est parcourable dans les deux sens : dans ce cas, elle produit deux arcs, l'un avec `reversed = false`, l'autre avec `reversed = true`.



— un attribut privé `final`, nommé `adjacency`, de type `List<List<Arc>>` pour représenter la liste des arcs sortants de chaque sommet.

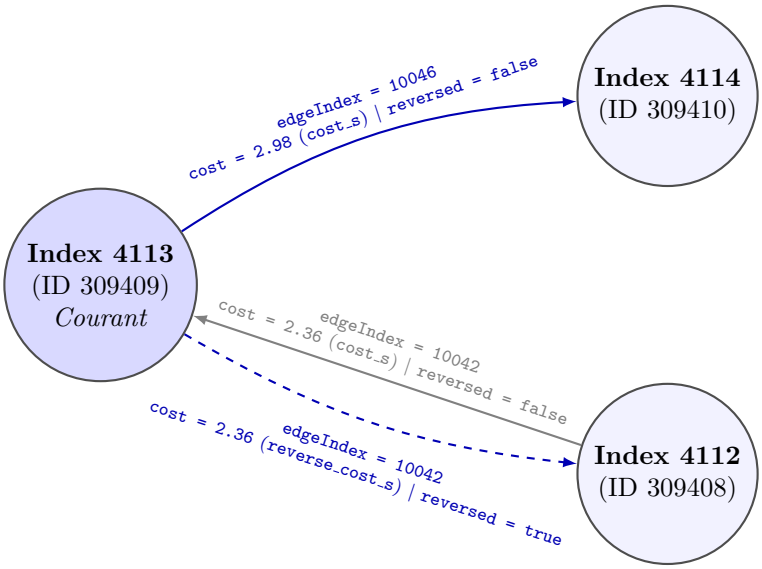


FIGURE 1 – Le sommet courant d'`Index 4113` et ses arcs sortants :
adjacency.get(4113) =
List.of(new Arc(**4112**, 2.36, 10042, true), new Arc(**4114**, 2.98, 10046, false))

Vérifiez la correspondance entre les arcs affichés dans le graphe de la figure 1 et les données CSV. La ligne CSV 10043 autorise les deux sens : `cost_s` décrit le trajet source → cible et `reverse_cost_s` le trajet cible → source. La ligne 10047 est à sens unique (`reverse_cost_s = -1`) : elle ne produit que l'arc source → cible.

```
stud@psl: ~/COURS/JAVA/potager/ $ awk -F';' '$2 == 309409 || $3 == 309409 {print $1";"$2";"$3";"$4";"$5}'
    data/arcs-enrichis.csv
10043;309408;309409;2.355705;2.355705
10047;309409;309410;2.980126;-1.000000
```

Expliquez le décalage entre les indices de tableau et les identifiants CSV.

Question 2 Une ligne brisée de k sommets est représentée par un tableau de $n = 2k$ coordonnées, nommé `geometry`, de la forme $\{x_0, y_0, x_1, y_1, \ldots, x_{k-1}, y_{k-1}\}$,

où chaque paire (x_i, y_i) représente un sommet de la ligne brisée (dans notre cas, un couple (lat, lon) produit à la question 4).
Après avoir observé les équivalences suivantes :

$$\begin{aligned} 0 \leq i \leq k-1 \Leftrightarrow \\ 0 \leq 2i \leq n-2 \Leftrightarrow \\ 0 \leq n-2-2i \leq n-2 \Leftrightarrow \\ 0 \leq 2(k-1-i) \leq n-2 \end{aligned}$$

$$\begin{aligned} 0 \leq i \leq k-1 \Leftrightarrow \\ 1 \leq 2i+1 \leq n-1 \Leftrightarrow \\ 1 \leq n-2i-1 \leq n-1 \Leftrightarrow \\ 1 \leq 2(k-i-1)+1 \leq n-1 \end{aligned}$$

définissez

a) une involution de $\{0, 2, \ldots, 2k-2\}$,
b) une involution de $\{1, 3, \ldots, 2k-1\}$.

Déduisez de ces involutions une méthode, nommée `reverse`, de signature

```
private static double[] reverse(double[] geometry)
```

qui renvoie un **nouveau** tableau représentant la ligne brisée d'orientation inverse, sans modifier le tableau `geometry` reçu en argument.
Cela signifie que la fonction `reverse` qui prend en argument un tableau de la forme $\boldsymbol{\{a_0, a_1, a_2, a_3, \ldots, a_{n-2}, a_{n-1}\}}$ devra renvoyer le tableau suivant :

$$\boxed{\boldsymbol{\{a_{n-2}, a_{n-1}, \ldots, a_2, a_3, a_0, a_1\}}}.$$

Question 3. Complétez la méthode `readVertices` en remplaçant les occurrences de `???` de sorte que la méthode

```
private void readVertices(Path file)
```

applique la procédure suivante :
pour chaque enregistrement `[id, x, y]` d'un sommet extrait du fichier `data/sommets-enrichis.csv`
— convertit les coordonnées Lambert93 `(x,y)` en coordonnées WGS84 `(lat, lon)` via `(lat,lon) = Lambert93.toWgs84(x, y)`,

— crée l'association $ID \rightarrow index$ dans `indexByNodeId`, l'index étant la position que le sommet va occuper dans les listes,
— ajoute son identifiant dans `nodeIds`,
— ajoute `lat` dans `latitudes` et `lon` dans `longitudes`,
— initialise une liste vide dans `adjacency` pour préparer l'ajout des arcs.

```
public final class RoadGraph {

  public record Arc(int targetIndex, double cost, int edgeIndex, boolean reversed) {}

  private final Map<Integer, Integer> indexByNodeId = new HashMap<>();
  private final List<Integer> nodeIds = new ArrayList<>();
  private final List<Double> latitudes = new ArrayList<>();
  private final List<Double> longitudes = new ArrayList<>();
  private final List<List<Arc>> adjacency = new ArrayList<>();

  private final List<double[]> edgeGeometries = new ArrayList<>();
  private final List<int[]> edgeEndpoints = new ArrayList<>();
  private final List<double[]> edgeCosts = new ArrayList<>();
  private int realEdgeCount;

  private static final double METERS_PER_DEGREE = 111_320.0;
  private static final double WALKING_SPEED = 1.2;

  private RoadGraph() {}

  public static RoadGraph load(Path verticesCsv, Path edgesCsv) {
    RoadGraph graph = new RoadGraph();
    graph.readVertices(verticesCsv);
    graph.readEdges(edgesCsv);
    return graph;
  }

  private void readVertices(Path file) {
    try (BufferedReader reader = Files.newBufferedReader(file, StandardCharsets.[???])) {
      reader.readLine();
      String line;
      while ((line = reader.readLine()) != null) {
        if (line.isBlank()) {
          [???];
        }
        String[] columns = line.split("[???]");
        int id = Integer.parseInt(columns[[???]].trim());
        double[] latLon = Lambert93.toWgs84(
            Double.parseDouble(columns[1].trim()),
            Double.parseDouble(columns[2].trim()));
        indexByNodeId.put(id, [???]);
        nodeIds.add(id);
        latitudes.add(latLon[0]);
        longitudes.add(latLon[1]);
        adjacency.add(new ArrayList<>());
      }
    } catch (IOException e) {
      throw new UncheckedIOException("Lecture impossible : " + file, e);
    }
  }

}
```

Question 4. Implémentez `private double[] parseGeometry(String wkt, int sourceIndex, int targetIndex)` qui :

— nettoie la chaîne WKT pour ne conserver que les paires de coordonnées,

— convertit chaque paire de Lambert 93 vers WGS84,
— retourne un tableau aplati de la forme

`[lat1, lon1, lat2, lon2, ...]`,

— inverse le tableau si la géométrie WKT a été numérisée dans le sens inverse (de la cible vers la source).

On pourra compléter le code `Java` fourni ci-dessous en remplaçant les occurrences de `???` par des instructions adéquates.

```java
private static double parseCost(String raw) {
   String value = raw.trim();
   if (value.isEmpty())
     return -1;
   double cost = Double.parseDouble(value);
   return Double.isFinite(cost) && cost >= 0 ? cost : -1;
 }

 private boolean isCloserToEnd(double[] geometry, int sourceIndex) {
   double sourceLat = latitudes.get(sourceIndex);
   double sourceLon = longitudes.get(sourceIndex);
   double toStart = Math.hypot(geometry[0] - sourceLat, geometry[1] - sourceLon);
   // Math.hypot calcule la distance euclidienne de façon optimisée
   // en gérant les risques de débordement.
   double toEnd = Math.hypot(geometry[geometry.length - 2] - sourceLat, geometry[geometry.length - 1]
    - sourceLon);
   return toEnd < toStart;
 }

 /**
  * WKT : suite de couples (x y) en Lambert 93 ;
  * resultat : suite de couples (lat, lon) en WGS84, orientee source vers target
 */

 private double[] parseGeometry(String wkt, int sourceIndex, int targetIndex) {
   String cleaned = wkt.replace("\"", "");
   int start = cleaned.indexOf('(');
   if (start < 0) {
     return new double[] {
         latitudes.get(sourceIndex), longitudes.get(sourceIndex),
         latitudes.get(targetIndex), longitudes.get(targetIndex) };
   }
   String body = cleaned.substring(start).replace("(", " ").replace("???", " ");
   String[] pairs = body.split("???");
   List<Double> coordinates = new ArrayList<>(pairs.length * 2);
   for (String pair : pairs) {
     String[] parts = pair.trim().split("\\s+");
     if (parts.length < 2) {
       ???;
     }
     double[] latLon = Lambert93.toWgs84( Double.parseDouble(parts[0]), Double.parseDouble(parts[1])
    );

     coordinates.add(latLon[0]);
     coordinates.add(latLon[1]);
   }
   double[] geometry = new double[coordinates.size()];
   for (int i = 0; i < geometry.length; i++) {
     geometry[i] = coordinates.get(i);
   }
   if (geometry.length >= 4 && isCloserToEnd(geometry, sourceIndex)) {
     geometry = ???(geometry);
   }
   return geometry;
 }
```

Remarque : dans les fichiers fournis, toutes les géométries sont déjà orientées de la source vers la cible ; l'inversion par `isCloserToEnd` ne se déclenche donc jamais. Pour la tester, appelez `parseGeometry` sur une chaîne WKT dont vous aurez inversé l'ordre des points.

Question 5. Complétez la méthode `readEdges` en remplaçant les occurrences de `???` de sorte que la méthode

`private void readEdges(Path file)`

applique la procédure suivante :

— lit un enregistrement du fichier `arcs-enrichis.csv`,
— traduit les identifiants *source* et *cible* en indices locaux (l'enregistrement est ignoré si l'un des sommets est introuvable),
— lit les coûts,
— génère la géométrie et stocke les attributs de l'arête,
— construit le graphe orienté : si le coût aller est ≥ 0, ajoute un `Arc` à la source avec `reversed = false` ; si le coût retour est ≥ 0, ajoute un `Arc` à la cible avec `reversed = true`.

À la fin de la lecture, `realEdgeCount` mémorise le nombre d'arêtes issues du fichier : les arêtes ajoutées ensuite (sommets virtuels, raccordements piétons) auront un `edgeIndex` supérieur ou égal à cette valeur.

```
private void readEdges(Path file) {
    try (BufferedReader reader = Files.newBufferedReader(file,
    StandardCharsets.UTF_8)) {
      reader.readLine();
      String line;
      while ((line = reader.readLine()) != null) {
        if (line.isBlank()) {
          ???;
        }
        String[] columns = line.split(";", 6);
        if (columns.length < 6) {
          continue;
        }
        Integer source =
    indexByNodeId.get(Integer.parseInt(columns[???].trim()));
        Integer target =
    indexByNodeId.get(Integer.parseInt(columns[???].trim()));
        if (source == null || target == null) {
          ???;
        }
        double forwardCost = parseCost(columns[???]);
        double reverseCost = parseCost(columns[???]);
        double[] geometry = parseGeometry(columns[5], source, ???);

        int edgeIndex = edgeGeometries.size();
        edgeGeometries.add(geometry);
        edgeEndpoints.add(new int[] { source, target });
        edgeCosts.add(new double[] { forwardCost, reverseCost });
        if (forwardCost >= 0) {
          adjacency.get(source).add(new Arc(target, forwardCost, edgeIndex,
    ???));
        }
        if (reverseCost >= 0) {
          adjacency.get(???).add(new Arc(source, reverseCost, edgeIndex,
    ???));
        }
      }
      realEdgeCount = edgeGeometries.size();
    } catch (IOException e) {
      throw new UncheckedIOException("Lecture impossible : " + file, e);
    }
  }
```